

SQL Data Analysis Project: Maven Movies

By Vivek Kumar

Introduction

I conducted this project to analyze rental trends, customer behavior, and revenue patterns using SQL queries. The project is based on the Maven Movies dataset, which contains information about films, customers, rentals, and payments. Through this project, I have applied various **SQL techniques, including Joins, Window Functions, Common Table Expressions (CTEs), and Aggregations**, to extract meaningful insights from the data.

Objective

- **To analyze customer behavior, rental trends, and film performance using the Maven Movies dataset.**
- Apply Joins, CTEs, Aggregations, Window Functions, and Recursive Queries for meaningful insights.
- Improve query readability, optimize performance, and explore business strategies.

02

Expected Insights

- ◆ The top rented films belong to the **Action and Comedy genres**.
- ◆ **Customers in major cities** tend to rent more frequently.
- ◆ Revenue peaks occur in specific months, **highlighting seasonal trends**.
- ◆ **Certain customers contribute significantly to total revenue**, indicating potential VIP customers.

01. Functions and GROUP BY Analysis

02. Joins for Data Retrieval

03. Window Functions for Ranking & Trends

Dataset Overview



Table Name	Description
<i>film</i>	Contains movie details (title, length, rental rate, rating, etc.)
<i>actor</i>	Stores actor names and their IDs
<i>film_actor</i>	Links films to actors (many-to-many relationship)
<i>category</i>	Stores movie genres (e.g., Action, Comedy, Drama)
<i>film_category</i>	Links films to categories
<i>inventory</i>	Represents copies of movies available in stores
<i>rental</i>	Stores rental transactions (who rented what and when)
<i>customer</i>	Contains customer details (name, email, address)
<i>payment</i>	Tracks payments made by customers
<i>staff</i>	Stores employee details
<i>store</i>	Represents different store locations

01

Dataset Acquisition

- Downloaded the Maven Movies dataset from GitHub.

02

Understanding the Dataset & ERD

- Analyzed table structures and relationships using an Entity-Relationship Diagram (ERD).

03

Data Cleaning & Transformation

- Ensured data consistency, handled missing values, and optimized table structures for querying.

04

Querying the Questions

- Designed and executed SQL queries to extract meaningful insights using CTEs, Joins, Aggregations, and Window Functions.

05

Generating Insights

- Interpreted query results to uncover key trends and patterns in movie rentals, customer behavior, and revenue.

06

Documentation & Presentation

- Compiled findings, queries, and insights into a well-structured document and presentation for knowledge sharing.

Project map



Queries

```
-- Question 1--  
-- Retrieve the total number of rentals made in the Sakila database.
```

```
SELECT COUNT(RENTAL_ID) AS TOTAL_RENTALS  
FROM RENTAL
```

Result Grid				Filter Rows
	TOTAL_RENTALS			
▶	16044			

```
-- Question 2:  
-- Find the average rental duration (in days) of movies rented from the Sakila database
```

```
SELECT AVG(DATEDIFF(LAST_UPDATE, RENTAL_DATE)) AS AVG_RENTAL  
FROM RENTAL
```

Result Grid				Filter Rows
	AVG_RENTAL			
▶	207.1577			

```
-- 3. Display the first name and last name of customers in uppercase.
```

```
SELECT UPPER(CONCAT(FIRST_NAME, ' ', LAST_NAME)) AS CUSTOMER_NAME  
FROM CUSTOMER
```

Result Grid				Filter Rows:
	CUSTOMER_NAME			
▶	MARY SMITH			
	PATRICIA JOHNSON			

Queries

-- Question 4:

-- Extract the month from the rental date and display it alongside the rental ID.

```
SELECT RENTAL_ID, MONTH(RENTAL_DATE)
FROM RENTAL
```

-- Question 5:

-- Retrieve the count of rentals for each customer (display customer ID and the count of rentals).

```
SELECT CUSTOMER_ID, COUNT(RENTAL_ID) AS RENTAL_COUNT
FROM RENTAL
GROUP BY CUSTOMER_ID
```

Result Grid			Filter Rows:
	CUSTOMER_ID	COUNT(RENTAL_ID)	
▶	1	32	
	2	27	
	3	26	

-- Question 6:

-- Find the total revenue generated by each store.

```
SELECT S.STORE_ID, SUM(P.AMOUNT) AS REVENUE
FROM CUSTOMER C
JOIN PAYMENT P ON C.CUSTOMER_ID = P.CUSTOMER_ID
JOIN STORE S ON S.STORE_ID = C.STORE_ID
GROUP BY S.STORE_ID
```

Result Grid			Filter Rows:
	STORE_ID	REVENUE	
▶	1	37001.52	
	2	30414.99	

Queries

```
-- -- Question 7:
-- Determine the total number of rentals for each category of movies --

SELECT C.NAME AS CATEGORY_NAME, COUNT(R.RENTAL_ID) AS RENTAL_NO FROM
CATEGORY C
JOIN FILM_CATEGORY FC ON C.CATEGORY_ID = FC.CATEGORY_ID
JOIN INVENTORY I ON I.FILM_ID = FC.FILM_ID
JOIN RENTAL R ON R.INVENTORY_ID = I.INVENTORY_ID
GROUP BY C.NAME
ORDER BY RENTAL_NO DESC;
```

```
-- Question 8:
-- Find the average rental rate of movies in each language.
```

```
SELECT L.NAME, AVG(F.RENTAL_RATE) AS AVG_RATE
FROM FILM F
JOIN LANGUAGE L ON F.LANGUAGE_ID = L.LANGUAGE_ID
GROUP BY L.NAME;
```

Result Grid			Filter Rows:
	CATEGORY_NAME	RENTAL_NO	
▶	Sports	1179	
	Animation	1166	
	Action	1112	
	Sci-Fi	1101	

Queries

```
-- Questions 9 -
```

```
-- Display the title of the movie, customer s first name, and last name who rented it.
```

```
-- Hint: Use JOIN between the film, inventory, rental, and customer tables.
```

```
SELECT F.TITLE, C.FIRST_NAME, C.LAST_NAME
FROM FILM F
JOIN INVENTORY I ON F.FILM_ID = I.FILM_ID
JOIN RENTAL R ON R.INVENTORY_ID = I.INVENTORY_ID
JOIN CUSTOMER ON C.CUSTOMER_ID = R.CUSTOMER_ID
```

```
-- Question 10:
```

```
-- Retrieve the names of all actors who have appeared in the film "Gone with the Wind."
```

```
SELECT A.FIRST_NAME, A.LAST_NAME
FROM FILM_ACTOR FA
JOIN FILM F ON FA.FILM_ID = F.FILM_ID
JOIN ACTOR A ON A.ACTOR_ID = FA.ACTOR_ID
WHERE F.TITLE = 'GONE WITH THE WIND';
```

Queries

-- Question 11:

-- Retrieve the customer names along with the total amount they've spent on rentals.

```
SELECT CONCAT(FIRST_NAME, ' ', LAST_NAME) AS C_NAME, SUM(AMOUNT) AS AMOUNT
FROM CUSTOMER C
JOIN PAYMENT P ON C.CUSTOMER_ID = P.CUSTOMER_ID
JOIN RENTAL R ON R.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY C.CUSTOMER_ID, C_NAME -- Grouping by customer ID ensures accuracy
```

Result Grid					Filter Rows:	
	C_NAME	AMOUNT				
▶	MARY SMITH	3797.76				
	PATRICIA JOHNSON	3475.71				
	LINDA WILLIAMS	3529.24				
	BARBARA JONES	1799.16				

-- Question 12:

-- List the titles of movies rented by each customer in a particular city (e.g., 'London').

-- Hint: JOIN customer, address, city, rental, inventory, and film tables, then use GROUP BY.

```
SELECT CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS C_NAME, F.TITLE, CI.CITY
FROM CUSTOMER C
JOIN ADDRESS A ON C.ADDRESS_ID = A.ADDRESS_ID
JOIN CITY CI ON A.CITY_ID = CI.CITY_ID
JOIN RENTAL R ON R.CUSTOMER_ID = C.CUSTOMER_ID
JOIN INVENTORY I ON I.INVENTORY_ID = R.INVENTORY_ID
JOIN FILM F ON F.FILM_ID = I.FILM_ID
WHERE CI.CITY = 'London' --
GROUP BY C.CUSTOMER_ID, F.TITLE, CI.CITY; --
```

Result Grid					Filter Rows:		Export
	C_NAME	TITLE	CITY				
▶	MATTIE HOFFMAN	CONQUERER NUTS	London				
	MATTIE HOFFMAN	WRATH MILE	London				

Queries

```
-- Question 13:  
-- Display the top 5 rented movies along with the number of times they've been rented.  
-- Hint: JOIN film, inventory, and rental tables, then use COUNT () and GROUP BY, and limit the results.
```

```
SELECT F.TITLE, COUNT(R.RENTAL_ID) AS RENTAL_COUNT  
FROM FILM F  
JOIN INVENTORY I ON F.FILM_ID = I.FILM_ID  
JOIN RENTAL R ON R.INVENTORY_ID = I.INVENTORY_ID  
GROUP BY F.TITLE  
ORDER BY RENTAL_COUNT DESC  
LIMIT 5;
```

Result Grid					Filter Rows:
	TITLE	RENTAL_COUNT			
▶	BUCKET BROTHERHOOD	34			
	ROCKETEER MOTHER	33			
	FORWARD TEMPLE	32			

```
-- Question 14:  
-- Determine the customers who have rented movies from both stores (store ID 1 and store ID 2).  
-- Hint: Use JOINS with rental, inventory, and customer tables and consider COUNT() and GROUP BY.
```

```
SELECT C.CUSTOMER_ID, COUNT(CONCAT(C.FIRST_NAME, ',', C.LAST_NAME)) AS CUSTOMER_NAME  
FROM RENTAL R  
JOIN INVENTORY I ON R.INVENTORY_ID = I.INVENTORY_ID  
JOIN CUSTOMER C ON C.CUSTOMER_ID = R.CUSTOMER_ID  
WHERE I.STORE_ID IN (1,2)  
GROUP BY C.CUSTOMER_ID  
HAVING COUNT(DISTINCT I.STORE_ID) = 2;
```

Result Grid					Filter Rows:
	CUSTOMER_ID	CUSTOMER_NAME			
▶	1	32			
	2	27			
	3	26			

Windows Queries

```
-- 1. Rank the customers based on the total amount they've spent on rentals.
```

```
SELECT A.CUSTOMER, A.AMOUNT_SPENT, DENSE_RANK() OVER( ORDER BY A.AMOUNT_SPENT DESC) AS RANKING
FROM
(SELECT CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER, SUM(P.AMOUNT) AS AMOUNT_SPENT
FROM RENTAL R
JOIN PAYMENT P ON R.RENTAL_ID = P.RENTAL_ID
JOIN CUSTOMER C ON C.CUSTOMER_ID = P.CUSTOMER_ID
GROUP BY C.FIRST_NAME, C.LAST_NAME
) A
```

Result Grid |   Filter Rows: | E

	CUSTOMER	AMOUNT_SPENT	RANKING
	KARL SEAL	221.55	1
	ELEANOR HUNT	216.54	2
	CLARA SHAW	195.58	3
	BLONDA KENNEDY	104.61	4

```
-- 2. Calculate the cumulative revenue generated by each film over time.
```

```
SELECT
    F.TITLE,
    P.PAYMENT_DATE,
    SUM(P.AMOUNT) OVER (PARTITION BY F.TITLE ORDER BY P.PAYMENT_DATE) AS CUMULATIVE_REVENUE
FROM PAYMENT P
JOIN RENTAL R ON P.RENTAL_ID = R.RENTAL_ID
JOIN INVENTORY I ON R.INVENTORY_ID = I.INVENTORY_ID
JOIN FILM F ON I.FILM_ID = F.FILM_ID
ORDER BY F.TITLE, P.PAYMENT_DATE;
```

Result Grid |   Filter Rows: | Export:  | Wrap C

	TITLE	PAYMENT_DATE	CUMULATIVE_REVENUE
▶	ACADEMY DINOSAUR	2005-05-27 07:03:28	0.99
	ACADEMY DINOSAUR	2005-05-30 20:21:07	2.98
	ACADEMY DINOSAUR	2005-06-15 02:57:51	3.97
	ACADEMY DINOSAUR	2005-06-17 20:24:00	4.96

Windows Queries

```
-- 3. Determine the average rental duration for each film, considering films with similar lengths.
```

```
SELECT
    F.TITLE,
    F.LENGTH,
    F.RENTAL_DURATION,
    AVG(F.RENTAL_DURATION) OVER (PARTITION BY F.LENGTH) AS AVG_RENTAL_DURATION
FROM FILM F;
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

TITLE	LENGTH	RENTAL_DURATION	AVG_RENTAL_DURATION
ALIEN CENTER	46	5	5.2000
IRON MOON	46	7	5.2000
KWAI HOMEWARD	46	5	5.2000
LABYRINTH LEAGUE	46	6	5.2000

```
-- 4. Identify the top 4 films in each category based on their rental counts.
```

```
SELECT TITLE, NAME AS CATEGORY_NAME, RENTAL_COUNT, RENTAL_RANK
FROM (
    SELECT
        F.TITLE,
        C.NAME,
        COUNT(R.RENTAL_ID) AS RENTAL_COUNT,
        DENSE_RANK() OVER(PARTITION BY C.NAME ORDER BY COUNT(R.RENTAL_ID) DESC) AS RENTAL_RANK
    FROM FILM F
    JOIN FILM_CATEGORY FC ON F.FILM_ID = FC.FILM_ID
    JOIN CATEGORY C ON C.CATEGORY_ID = FC.CATEGORY_ID
    JOIN INVENTORY I ON I.FILM_ID = F.FILM_ID
    JOIN RENTAL R ON R.INVENTORY_ID = I.INVENTORY_ID
    GROUP BY F.TITLE, C.NAME
) A
WHERE RENTAL_RANK <= 4;
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	TITLE	CATEGORY_NAME	RENTAL_COUNT	RENTAL_RANK
▶	RUGRATS SHAKESPEARE	Action	30	1
	SUSPECTS QUILLS	Action	30	1
	HANDICAP BOONDOCK	Action	28	2
	STORY SIDE	Action	28	2

Windows Queries

```
-- 5. Calculate the difference in rental counts between each customer's total rentals and the average rentals  
-- across all customers.
```

```
WITH CustomerRentalCount AS (  
    SELECT  
        C.CUSTOMER_ID,  
        CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME,  
        COUNT(R.RENTAL_ID) AS TOTAL_RENTALS  
    FROM rental R  
    JOIN customer C ON R.customer_id = C.customer_id  
    GROUP BY C.CUSTOMER_ID, CUSTOMER_NAME  
)
```

```
SELECT  
    CUSTOMER_ID,  
    CUSTOMER_NAME,  
    TOTAL_RENTALS,  
    ROUND(TOTAL_RENTALS - AVG(TOTAL_RENTALS) OVER (), 2) AS RENTAL_DIFF  
FROM CustomerRentalCount  
ORDER BY RENTAL_DIFF DESC;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 				
	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_RENTALS	RENTAL_DIFF
▶	148	ELEANOR HUNT	46	19.22
	526	KARL SEAL	45	18.22
	144	CLARA SHAW	42	15.22
	236	MARCIA DEAN	42	15.22

Windows Queries

```
-- 6. Find the monthly revenue trend for the entire rental store over time.
```

```
SELECT
```

```
    YEAR(P.PAYMENT_DATE) AS YEARLY,
```

```
    MONTH(P.PAYMENT_DATE) AS MONTHLY,
```

```
    SUM(P.AMOUNT) AS TOTAL_REVENUE
```

```
FROM payment P
```

```
JOIN customer C ON P.customer_id = C.customer_id
```

```
JOIN store S ON C.store_id = S.store_id
```

```
GROUP BY YEARLY, MONTHLY
```

```
ORDER BY YEARLY, MONTHLY;
```

	YEARLY	MONTHLY	TOTAL_REVENUE
►	2005	5	4824.43
	2005	6	9631.88
	2005	7	28373.89

Windows Queries

```
-- 7. Identify the customers whose total spending on rentals falls within the top 28% of all customers.
```

```
WITH CustomerSpending AS (  
    SELECT  
        C.CUSTOMER_ID,  
        CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS NAME,  
        SUM(P.AMOUNT) AS TOTAL_SPEND  
    FROM payment P  
    JOIN customer C ON P.customer_id = C.customer_id  
    GROUP BY C.CUSTOMER_ID, NAME  
)  
, RankedCustomers AS (  
    SELECT *,  
        PERCENT_RANK() OVER (ORDER BY TOTAL_SPEND DESC) AS RANK_PERCENTILE  
    FROM CustomerSpending  
)
```

```
SELECT CUSTOMER_ID, NAME, TOTAL_SPEND  
FROM RankedCustomers  
WHERE RANK_PERCENTILE <= 0.28 -- Top 28% customers  
ORDER BY TOTAL_SPEND DESC;
```

	CUSTOMER_ID	NAME	TOTAL_SPEND
▶	526	KARL SEAL	221.55
	148	ELEANOR HUNT	216.54
	144	CLARA SHAW	195.58
	137	RHONDA KENNEDY	194.61
	133	MARTIN GIBBER	184.64

Windows Queries

```
-- 8. Calculate the running total of rentals per category, ordered by rental count.
```

```
WITH CategoryRentalCount AS (  
    SELECT  
        C.NAME AS CATEGORY,  
        COUNT(R.RENTAL_ID) AS RENTAL_COUNT  
    FROM rental R  
    JOIN inventory I ON R.inventory_id = I.inventory_id  
    JOIN film_category FC ON I.film_id = FC.film_id  
    JOIN category C ON FC.category_id = C.category_id  
    GROUP BY C.NAME  
)
```

```
SELECT  
    CATEGORY,  
    RENTAL_COUNT,  
    SUM(RENTAL_COUNT) OVER (ORDER BY RENTAL_COUNT DESC) AS RUNNING_TOTAL  
FROM CategoryRentalCount  
ORDER BY RENTAL_COUNT DESC;
```

	CATEGORY	RENTAL_COUNT	RUNNING_TOTAL
►	Sports	1179	1179
	Animation	1166	2345
	Action	1112	3457
	Sci-Fi	1101	4558

Conclusion & Learning Outcomes

- ✓ Learned how to use CTEs for better query structuring
- ✓ Explored real-world applications of Joins & Aggregations
- ✓ Applied Window Functions to rank & analyze data trends
- ✓ Understood how Recursive Queries help manage hierarchical data

06

Thank You